

Report of Assignment 3, Foundation of Machine Learning (DA5400)

Harshil Pandey, DA24C006

November 2024

Contents

1	Details of the Submission	1
2	Source Code and Implementation	2
2.1	Loading the Data	2
2.2	Plotting	2
2.3	Principal Component Analysis	3
2.4	K-Means Clustering	5
3	Observations and Inferences	7
3.1	PCA	7
3.2	KMeans	10

1 Details of the Submission

- Libraries Used:
 - `numpy`: for matrix and numerical operations
 - `pandas`: for loading and storing data in matrix format.
 - `matplotlib`: for plotting the graphs
 - `seaborn`: for plotting the graphs
 - `scikit-learn`: for loading the MNIST Dataset
- `models_helpers.py` file contains the source code which includes procedures that perform loading the data, visualization, and classes for two algorithms: K-Means Clustering and Principal Component Analysis(PCA).
- `demo.py` file contains the code to present the visualizations and run the algorithms on the provided dataset.

Before running any Python file, please ensure that the libraries mentioned above are installed.

2 Source Code and Implementation

As mentioned above, all the source code for the algorithms and helper functions is present in the `models_helpers.py` file.

2.1 Loading the Data

Data for running the PCA algorithm will be retrieved from `scikit-learn` library of Python, which has a function called `fetch_openml` which allows fetching the MNIST dataset. The dataset will be converted into a desirable format by the `load_data` helper function.

- `load_data(random_state=None)`: This function fetches the MNIST dataset such that each class has 100 random samples.

Parameters

`random_state` : integer for reproducible results

Returns

a tuple (`data`, `labels`) where

- `data` is a `numpy.ndarray` of shape (1000, 784) such that each row is a flattened image sample
- `labels` is `numpy.array` of 1000 corresponding labels.

Data for running the K-Means Algorithm will be loaded by using the `read_csv` function from the `pandas` library of Python.

2.2 Plotting

All plotting is achieved using the `matplotlib` and `seaborn` library of Python. Most of the plotting is done as needed without the use of helper functions. The only two helper functions used for plotting are:

- `plot_images(data, nrows=3, ncols=3, title='')`: This function plots the first `nrows*ncols` image samples in `data`.

Parameters

`data`: is a `numpy.ndarray` such that each row is a flattened image sample.

`nrows`: number of rows in the plot

`ncols`: number of columns in the plot

`title`: string that forms the title of each plot.

- `plot_for_pcs(nrows, ncols, image, for_pcs, model)`: This function plots the reconstructed image for an image using the different number of principal components.

Parameters

`nrows`: Number of rows in the plot

`ncols`: Number of columns in the plot

`image`: a `numpy.array` of the flattened representation of the image.

`for_pcs`: a list that specifies the number of principal components to use for that particular reconstruction

2.3 Principal Component Analysis

Principal Component Analysis is a dimensionality reduction technique that reduces the dimensionality of the data by decomposing it into the subspace spanned by several orthogonal components called Principal Components. If the data lies in a low dimensional subspace, then less number of Principal Components are needed to represent the data. Whereas, if the data lies in a higher dimensional subspace then representing it in a lower dimensional subspace incurs a lossy representation. As the number of Principal Components for representation is increased, the loss of data is reduced. The Principal Components are exactly the eigenvectors of the Covariance Matrix of the data. The variance captured by each Principal Component is the corresponding eigenvalue.

The algorithm is implemented using the `PCA` class. The instance variables are as follows:

- **means:** a `numpy.array` each component of which is the mean of the corresponding feature
- **evalues:** a `numpy.array` such that each component is an eigenvalue of the covariance matrix of the data. The eigenvalues are stored in the descending order.
- **evecutors:** a `numpy.ndarray` such that each column is the eigenvector of the covariance matrix of the data. Each eigenvector has a norm of 1. Moreover, the order of eigenvectors is the same as the order of corresponding eigenvalues stored in the `evalues` variable.

The class methods are as follows:

- **calc_cov_mat(self, X):** This function returns the covariance matrix of the data passed in X. The function mean centers the data before calculating the covariance matrix.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features. The covariance will be calculated for the columns.

Returns

the covariance matrix in the form of a `numpy.ndarray` of shape `(d, d)` where `d` is the number of features(columns) in X

The mean vector is calculated as:

$$\mu_j = \frac{1}{n} \sum_{i=1}^n X_{ij}$$

where, μ_j is the mean of the j^{th} feature, n is the number of sample, X_{ij} is the j^{th} feature of the i^{th} sample.

The mean-centered data matrix is calculated as:

$$\bar{X}_{ij} = X_{ij} - \mu_j$$

The covariance matrix C is such that the C_{pq} is the covariance between the p^{th} and q^{th} features of X . The precise formula is as follows:

$$\begin{aligned}
C_{pq} &= \frac{1}{n} \sum_{i=1}^n (X_{ip} - \mu_p)(X_{iq} - \mu_q) \\
&= \frac{1}{n} \sum_{i=1}^n \bar{X}_{ip} \bar{X}_{iq}
\end{aligned}$$

Therefore,

$$C = \frac{1}{n} \bar{X}^T \bar{X}$$

- **eigen_decompose(self, X)**: This function returns the eigenvector decomposition of the matrix **X**

Parameters

X: a square matrix in the form of a `numpy.ndarray`

Returns

a tuple (**vals**, **vecs**) where

- **vals** is the vector of eigenvalues of **X** in the same format as **evalues**
- **vecs** is the matrix of eigenvectors of **X** in the same format as **evectors**

In the algorithm, this function is used to find the eigenvector decomposition of the Covariance Matrix. Let the eigenvector matrix of C be W such that each column of W is a normalized eigenvector of C . The order of columns of W is the descending of order of the corresponding eigenvalues.

- **fit(self, X)**: This function calculates the principal components of **X** and stores in the corresponding instance variables.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features.

- **transform(self, X, use_pcs=None)**: This function reduces the dimensionality of the data passed by compressing the data into **use_pcs** number of features. For doing this the first **use_pcs** components are utilized, which are stored as columns of **evalues**.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features. The number of features in **X** must be the same as the dimension of the eigenvectors stored in **evectors**, i.e. the number of rows in **evectors**. **X** is mean-centered using **means** before transformation.

use_pcs: an integer that specifies the number of principal components to use for the transformation. If **use_pcs** is **None** then all principal components are used.

Returns

the transformed data matrix in a `numpy.ndarray` with the transformed data points as rows and columns as features.

The transformed representation is calculated as follows:

$$\hat{X}_{iq} = \sum_{j=1}^d \bar{X}_{ij} W_{jq}$$

where, $\hat{X}$ is the transformed data matrix.

It is easy to infer that the q^{th} component(feature) of the transformed representation of the i^{th} sample is the dot product of the mean-centered i^{th} sample and the q^{th} principal component. Therefore, the formula for complete calculation is:

$$\hat{X} = \bar{X}W$$

If the representation has to be calculated using only a few components, then only that many columns of W should be used to calculate $\hat{X}$.

- **reconstruct(self, X, use_pcs=None):** This function reconstructs the transformed data into the original dimension using the principal components. The calculated feature means are added to the reconstruction.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features. The number of features in **X** must be the same as the `use_pcs`.

use_pcs: an integer that specifies the number of principal components to use for the reconstruction. If `use_pcs` is 'None' then all principal components are used.

Returns

the reconstructed data matrix in a `numpy.ndarray` with the reconstructed data points as rows and columns as features.

Since W is an orthogonal matrix, the following is true:

$$\bar{X} = \hat{X}W^T$$

Analogously, if few components were used to transform the data, then only those components can be taken in the above formula in place of W to obtain the reconstruction. Finally, the mean vector is added to each reconstructed sample to obtain the result.

2.4 K-Means Clustering

K-Means Clustering is an iterative algorithm that assigns a single cluster to each sample based on the proximity to different cluster means. The algorithm is implemented using the `KMeans` class and the instance variables are as follows:

- **n_clusters:** the number of clusters to make
- **assignments:** the array that specifies which training data point is assigned to which cluster. The i^{th} element of the array is the cluster number assigned to the i^{th} training data point.
- **means:** a `numpy.ndarray` that specifies the means of the clusters. The array has a shape of `(n_clusters, 1, d)`, where `d` is the dimensionality of the training data.

- **errors**: array to keep track of the error in each iteration.
- **random_state**: an integer for reproducible initialization
- **max_iter**: the number of maximum iterations to run for the algorithm

The class methods are as follows:

- **initialize(self, X)**: This function picks cluster means uniformly at random from the dataset passed.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features.

- **assign_clusters(self, X)**: This function returns the cluster assignments for the data points based on the means learned.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features.

Returns

a `numpy.array` which is in the same format as **assignments**

The i^{th} sample is assigned to that cluster whose mean is the closest to it. Mathematically,

$$z_i = \operatorname{argmin}_k \{ ||x_i - \mu_k||^2 \}$$

where z_i is the cluster assigned to the i^{th} sample, x_i is the i^{th} sample, and μ_k is the mean of the k^{th} cluster center.

- **update_means(self, X)**: This function updates the means based on the cluster assignments.

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features.

Mathematically,

$$\mu_k = \frac{\sum_{i=1}^n x_i \mathbf{1}(z_i = k)}{\sum_{i=1}^n \mathbf{1}(z_i = k)}$$

- **error(self, X)**: This function calculates the error based on the means and cluster assignments

Parameters

X: a `numpy.ndarray` with data points as rows and columns as features.

Returns

the clustering error(float)

The formula for error is as follows:

$$Error = \sum_{i=1}^n ||x_i - \mu_{z_i}||^2$$

- `fit(self, X)`: This function runs the KMeans Clustering algorithm. A random initialization of means is done using the `initialize` function. Then, using the `assign_clusters` and `update_means` functions, the progress is made iteratively. In each iteration, the error is also calculated using the `error` function. If the assignments obtained in the current iteration are the same as previous assignments, then the algorithm is stopped.

Parameters

`X`: a `numpy.ndarray` with data points as rows and columns as features.

- `plot_clusters(self, X, clusters_colors)`: This function plots a graph with one subgraph being the error plot, and the other subgraph being a scatter plot of the cluster assignments.

Parameters

`X`: a `numpy.ndarray` with data points as rows and columns as features

`clusters_colors`: a list specifying the colors for different clusters.

- `plot_voronoi(self, X, cluster_colors)`: This function plots the voronoi regions for each cluster.

Parameters

`X`: a `numpy.ndarray` with data points as rows and columns as features

`clusters_colors`: a list specifying the colors for different clusters.

3 Observations and Inferences

3.1 PCA

Figure 1 gives a snapshot of the dataset. Each sample is a 28x28 image. Instead of directly dealing with the image we deal with the flattened image representation. So, each image is represented as a 784-dimensional vector.

The visualization of the principal components is presented in *Figure 2*. Each principal component is a 784-dimensional vector that has been visualized as an image. We can observe that the first few principal components are the most defined and show the general digit structure. The further we go, the principal components get unclear and noisy.

The variance explained by the first 5 components is mentioned below:

$$PC_1 : 3.434 \times 10^5$$

$$PC_2 : 2.449 \times 10^5$$

$$PC_3 : 2.214 \times 10^5$$

$$PC_4 : 1.959 \times 10^5$$

$$PC_5 : 1.741 \times 10^5$$

Figure 3 plots the percentage of explained variance as a function of the number of principal components. We can observe that the graph is steep for the first hundred principal components. As the number of principal components is increased, the increase in the explained variance is reduced, which reinforces the fact that the first few principal components explain the most variance in the data. We can also observe that almost all the variance in the data is explained by the first 300 principal components.

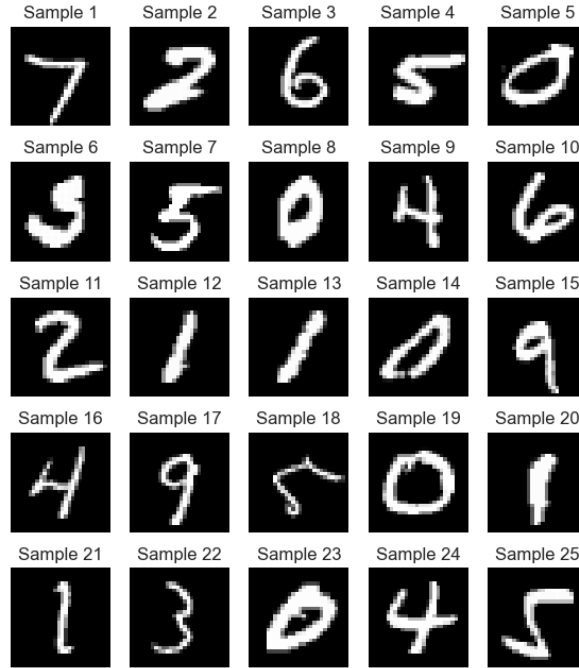


Figure 1: Original Samples

Reconstructing an image using different number of principal components gives an idea for choosing the suitable number of PCs for reconstruction. *Figure 4* does exactly that. We can observe that 300 or more principal components produce almost identical results when compared to the original image. If classification of digits is the downstream task, I would pick 300 principal components for representation as it captures almost all the variance and the pictures generated using 300 principal components are pretty much identical to the original.

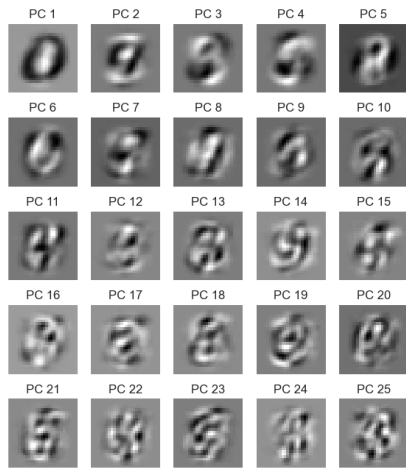


Figure 2: Principal Components

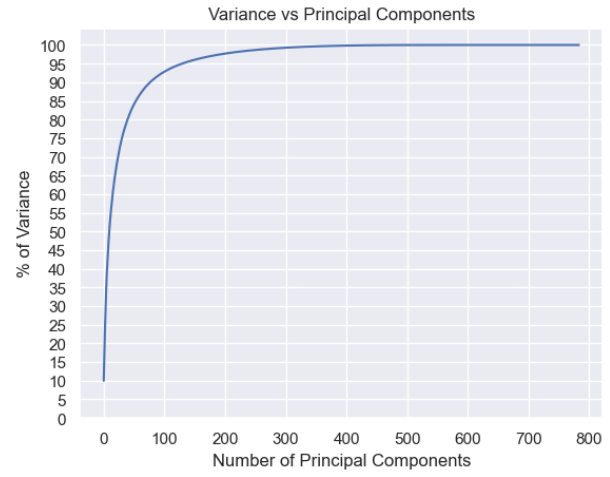


Figure 3: Explained Variance vs Principal Components

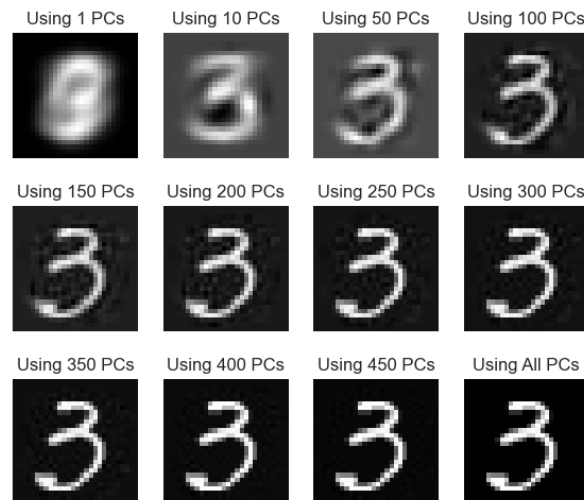


Figure 4: Reconstruction using different PCs

3.2 KMeans

After trying 5 random initializations for $k = 2$, the results are plotted from *Figure 5* to *Figure 9*. The Voronoi regions are plotted from *Figure 10* to *Figure 13*. Lloyd's Algorithm is not a good way to cluster this dataset because the natural clusters in the dataset are not convex. The cluster in the shape of a half-moon is not convex therefore, the KMeans algorithm fails to recognize these clusters properly. We can observe that the clusters are defined by their dense population in this case, so a method that makes use of the density of clusters will be able to recognize the clusters. One such technique is called the DBSCAN algorithm which clusters the data points based on their dense neighborhood. In this technique, a direct neighborhood of a data point is defined based on the Euclidean ball of some radius ϵ centered at that point. If a data point has more than or equal to a minimum number of other data points in its neighborhood, then it is termed as a core point. If a data point is not a core point but lies in the neighborhood of a core point, then it is called a border point. If a data point is neither a core point nor a border point, it is called a noise point. We start by identifying core, border, and noise points. Then we assign cluster numbers to the core points. A core point C_1 is said to be in the density neighborhood of another core point C_2 if there exists a third core point C_3 which is a direct neighbor of C_2 and C_1 is in the density neighborhood of C_3 . All core points in the same density neighborhood are assigned the same cluster. The border point is assigned the same cluster as its nearest core point, while the noise point is not assigned a cluster. The ϵ and minimum number of data points threshold can be treated as hyper-parameters for the algorithm. In this way, DBSCAN incorporates the density of the clusters and can identify non-convex clusters, hence being a good choice in this case.

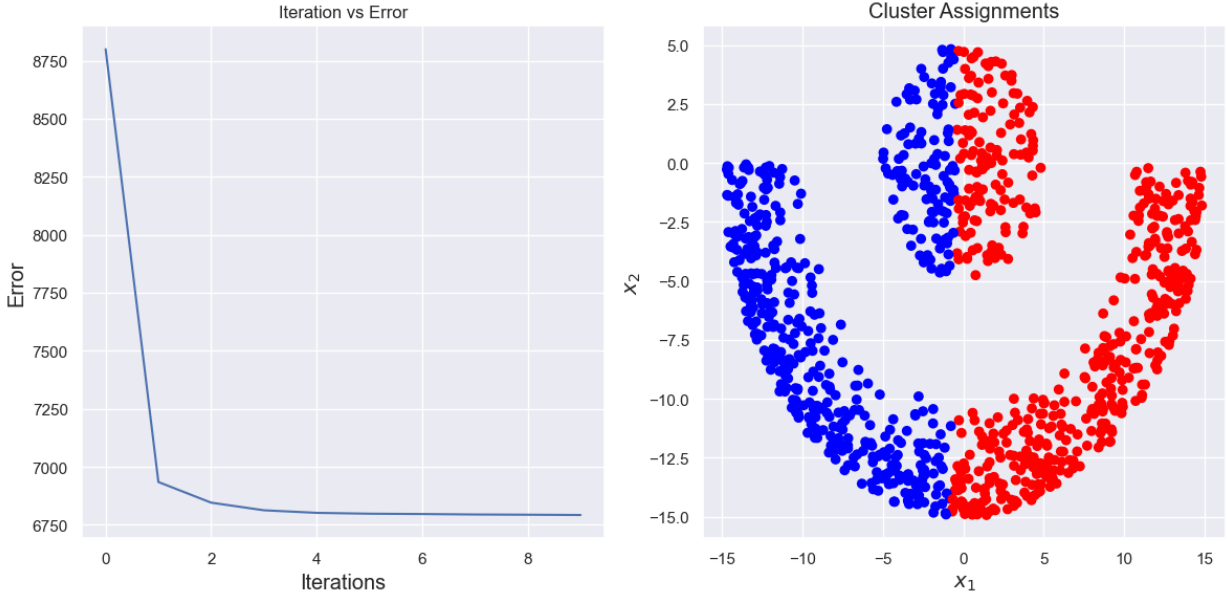


Figure 5: Initialization 1

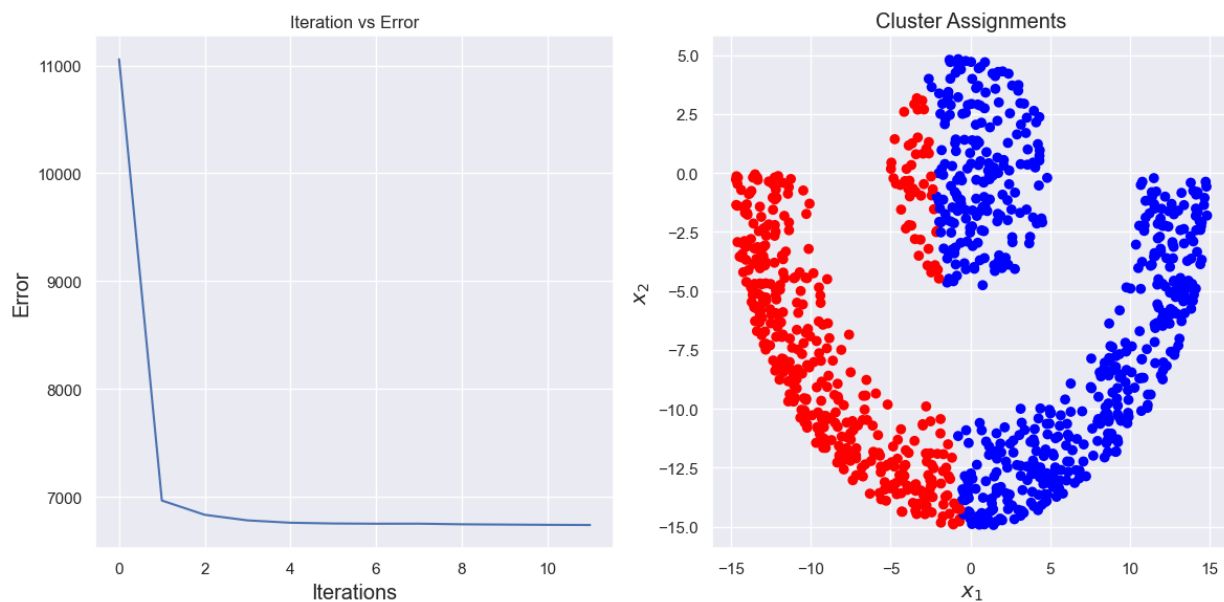


Figure 6: Initialization 2

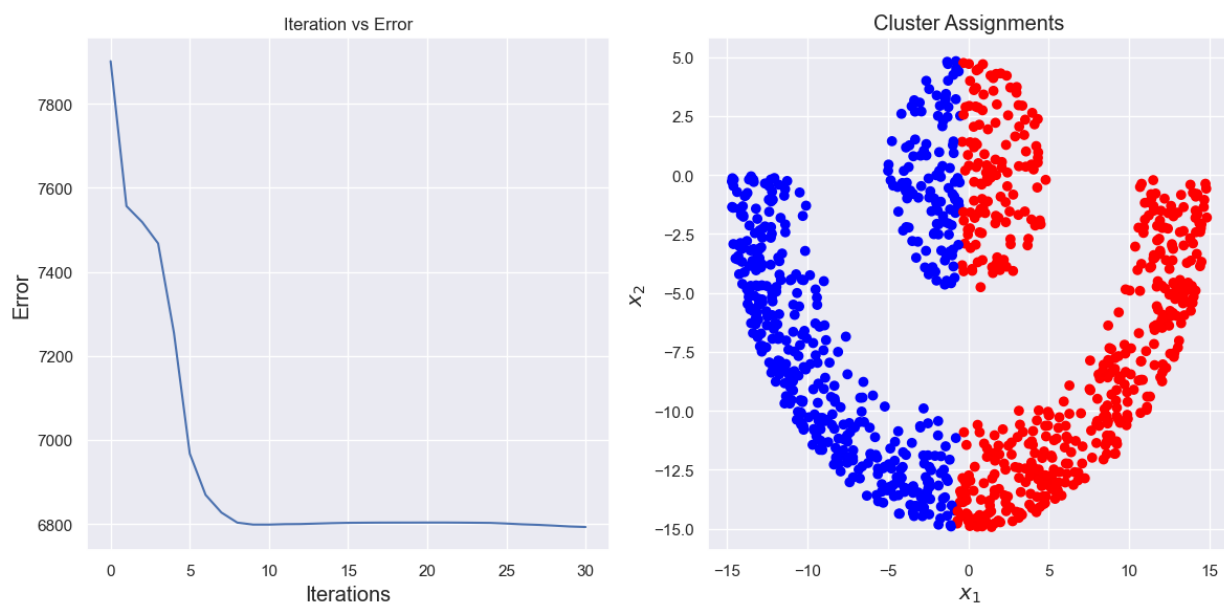


Figure 7: Initialization 3

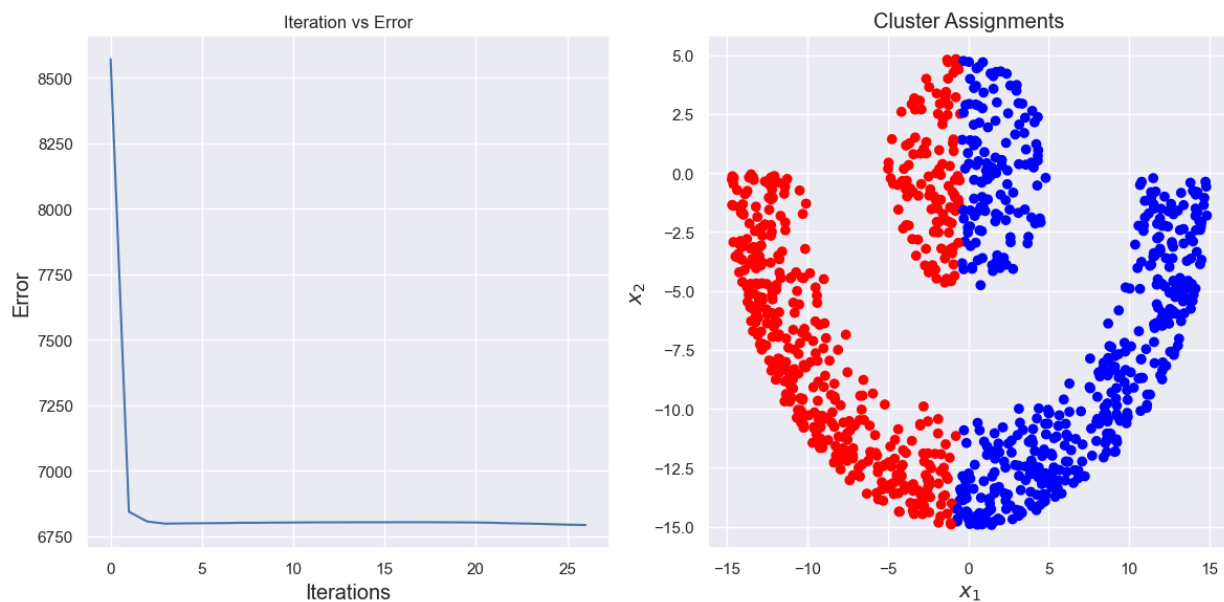


Figure 8: Initialization 4

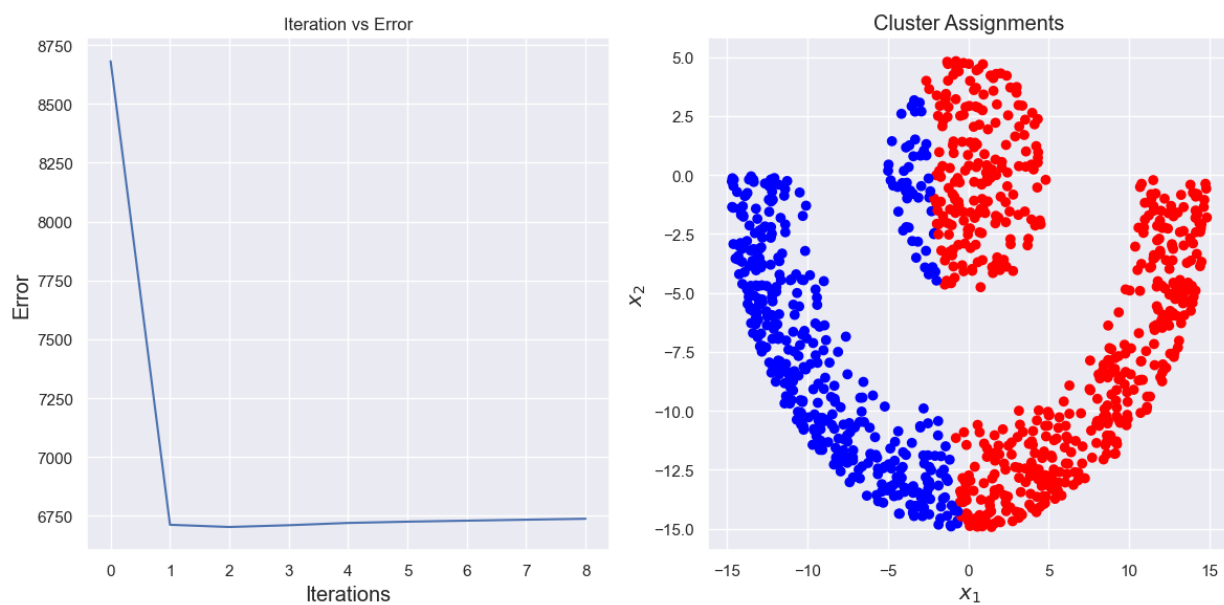


Figure 9: Initialization 5

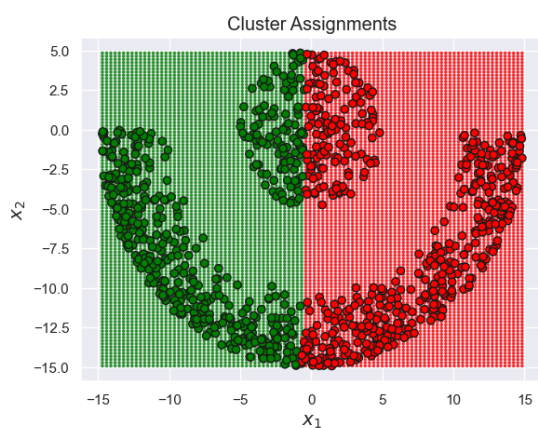


Figure 10: Voronoi Regions for $k = 2$

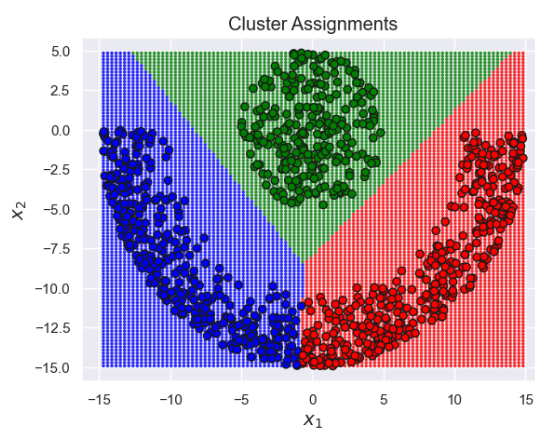


Figure 11: Voronoi Regions for $k = 3$

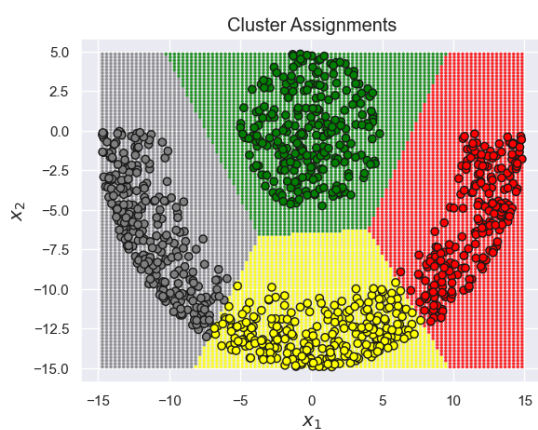


Figure 12: Voronoi Regions for $k = 4$

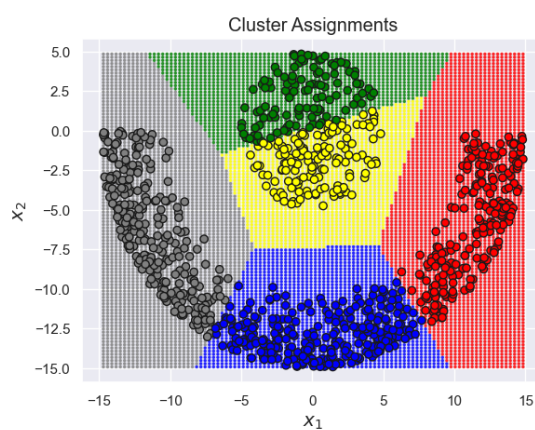


Figure 13: Voronoi Regions for $k = 5$